PS1 - IO

Tate Mason

```
# Packages
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(lubridate)
library(stargazer)
```

```
Please cite as:

 Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.
 R package version 5.2.3. https://CRAN.R-project.org/package=stargazer
```

```
library(fixest)


# Data
iri <- read_tsv("~/SchoolWork/Y2S1/IO/PS1/IRI.csv", show_col_types = FALSE)
names(iri) <- gsub("^t", "", names(iri))  # Clean names
iri <- iri %>%
```

```
  mutate(
    market_id = interaction(store_id, week_id, drop = TRUE),
    date_id = interaction(year, month, drop = TRUE),
    date = as.Date(paste(year, month, "01", sep = "-"))
  )
```

Question 1:

1.1

```
# Distinct counts per market
brands_per_market <- iri %>%
  distinct(market_id, brand) %>%
  count(market_id, name = "n_brands")

mfrs_per_market <- iri %>%
  distinct(market_id, parent) %>%
  count(market_id, name = "n_mfrs")

# Total market sales per market
total_sales_market <- iri %>%
  group_by(market_id) %>%
  summarise(total_sales = sum(quantity, na.rm = TRUE), .groups = "drop")

# Brand-level sales (market x brand)
brand_sales_market <- iri %>%
  group_by(market_id, brand) %>%
  summarise(brand_sales = sum(quantity, na.rm = TRUE), .groups = "drop")

# Per-market price & characteristics (means within market)
market_chars <- iri %>%
  group_by(market_id) %>%
  summarise(
    price_mean     = mean(price, na.rm = TRUE),
    fiber_mean     = mean(fiber, na.rm = TRUE),
    sugars_mean    = mean(sugars, na.rm = TRUE),
    flavored_mean  = mean(as.numeric(flavored), na.rm = TRUE),
    fortified_mean = mean(as.numeric(fortified), na.rm = TRUE),
    .groups = "drop"
```

```r
  )

# Helper to summarise numeric vectors
summarise_vec <- function(x) {
  tibble(
    mean   = mean(x, na.rm = TRUE),
    sd     = sd(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    max    = max(x, na.rm = TRUE)
  )
}

# Tables for 1.1
tbl_counts <- bind_cols(
  variable = c("n_brands", "n_mfrs"),
  bind_rows(
    summarise_vec(brands_per_market$n_brands),
    summarise_vec(mfrs_per_market$n_mfrs)
  )
)

tbl_total_sales <- bind_cols(
  variable = "total_sales (per market)",
  summarise_vec(total_sales_market$total_sales)
)

tbl_brand_sales <- bind_cols(
  variable = "brand_sales (per market-brand)",
  summarise_vec(brand_sales_market$brand_sales)
)

tbl_price_chars <- tribble(
  ~variable,                ~mean, ~sd, ~median, ~max,
  "price (market mean)",
    summarise_vec(market_chars$price_mean)$mean,
    summarise_vec(market_chars$price_mean)$sd,
    summarise_vec(market_chars$price_mean)$median,
    summarise_vec(market_chars$price_mean)$max,
  "fiber (market mean)",
    summarise_vec(market_chars$fiber_mean)$mean,
    summarise_vec(market_chars$fiber_mean)$sd,
    summarise_vec(market_chars$fiber_mean)$median,
```

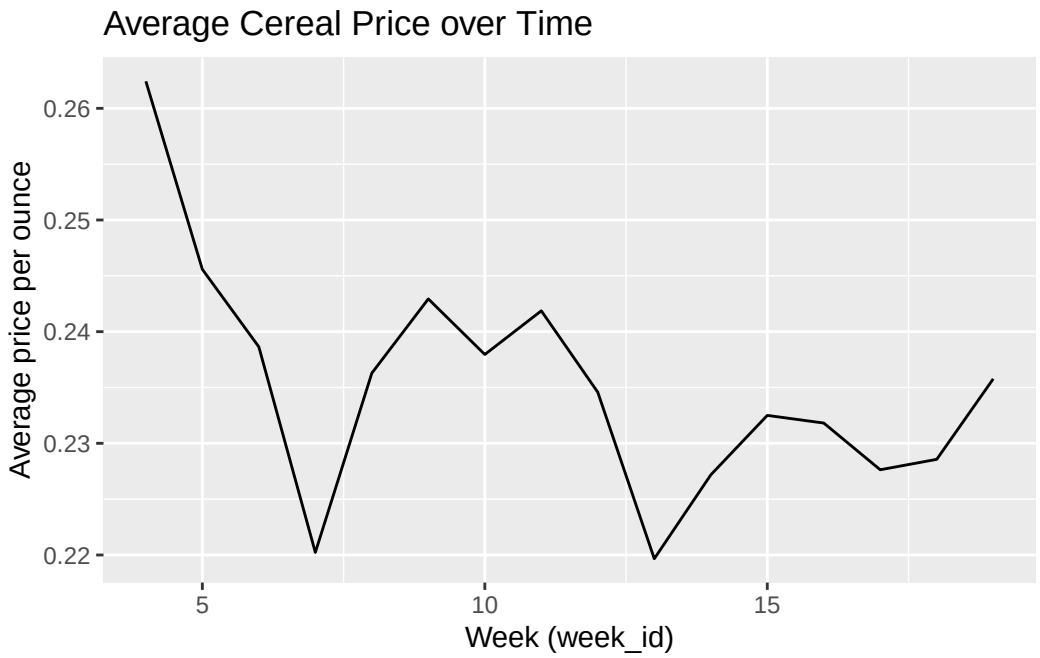
```
    summarise_vec(market_chars$fiber_mean)$max,
  "sugars (market mean)",
    summarise_vec(market_chars$sugars_mean)$mean,
    summarise_vec(market_chars$sugars_mean)$sd,
    summarise_vec(market_chars$sugars_mean)$median,
    summarise_vec(market_chars$sugars_mean)$max,
  "flavored share (market mean)",
    summarise_vec(market_chars$flavored_mean)$mean,
    summarise_vec(market_chars$flavored_mean)$sd,
    summarise_vec(market_chars$flavored_mean)$median,
    summarise_vec(market_chars$flavored_mean)$max,
  "fortified share (market mean)",
    summarise_vec(market_chars$fortified_mean)$mean,
    summarise_vec(market_chars$fortified_mean)$sd,
    summarise_vec(market_chars$fortified_mean)$median,
    summarise_vec(market_chars$fortified_mean)$max
)

summary_11 <- bind_rows(
  tbl_counts,
  tbl_total_sales,
  tbl_brand_sales,
  tbl_price_chars
)

summary_11
```

```
# A tibble: 9 x 5
  variable                          mean         sd    median       max
  <chr>                            <dbl>      <dbl>     <dbl>     <dbl>
1 n_brands                        34.9       4.82      37        39
2 n_mfrs                           3.95      0.259      4         4
3 total_sales (per market)     15555.    12358.     12557.    84567.
4 brand_sales (per market-brand)   446.      726.       238.    26470.
5 price (market mean)              0.235     0.0265     0.233     0.325
6 fiber (market mean)              1.30      0.0874     1.31      1.67
7 sugars (market mean)             8.31      0.205      8.33     10.2
8 flavored share (market mean)     0.640     0.0265     0.641     0.857
9 fortified share (market mean)    0.485     0.0341     0.487     0.667
```

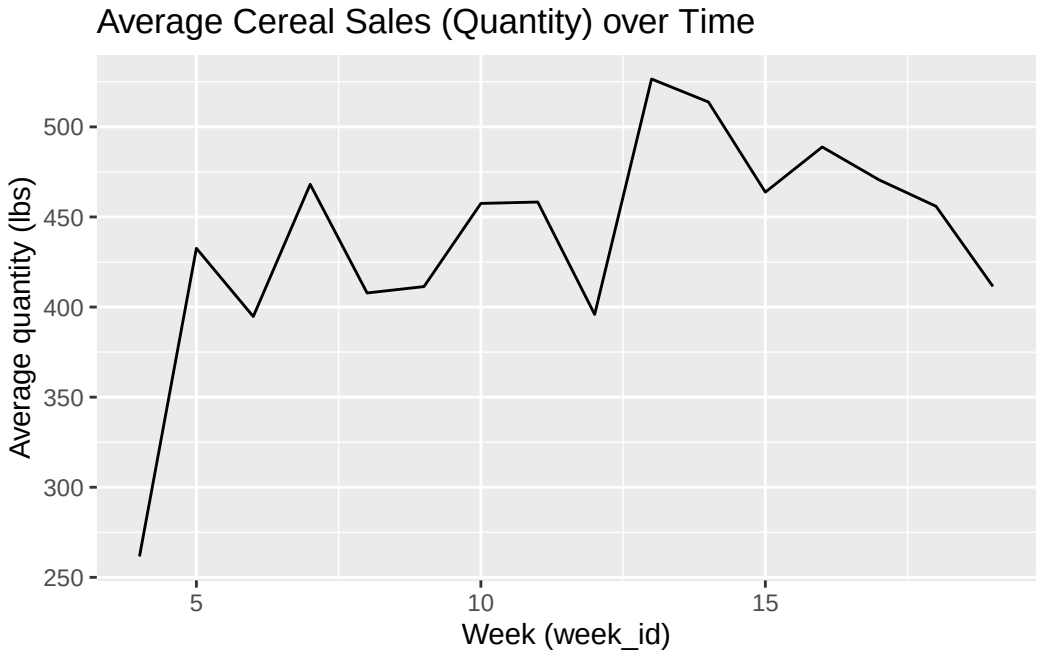
```
# If you want a file:
# write_csv(summary_11, "summary_11_descriptives.csv")
```

1.2

```
ts_simple <- iri %>%
  group_by(week_id) %>%
  summarise(
    avg_price = mean(price, na.rm = TRUE),
    avg_qty   = mean(quantity, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(week_id)

p_price <- ggplot(ts_simple, aes(x = week_id, y = avg_price)) +
  geom_line() +
  labs(title = "Average Cereal Price over Time",
       x = "Week (week_id)", y = "Average price per ounce")

p_qty <- ggplot(ts_simple, aes(x = week_id, y = avg_qty)) +
  geom_line() +
  labs(title = "Average Cereal Sales (Quantity) over Time",
       x = "Week (week_id)", y = "Average quantity (lbs)")

p_price
```

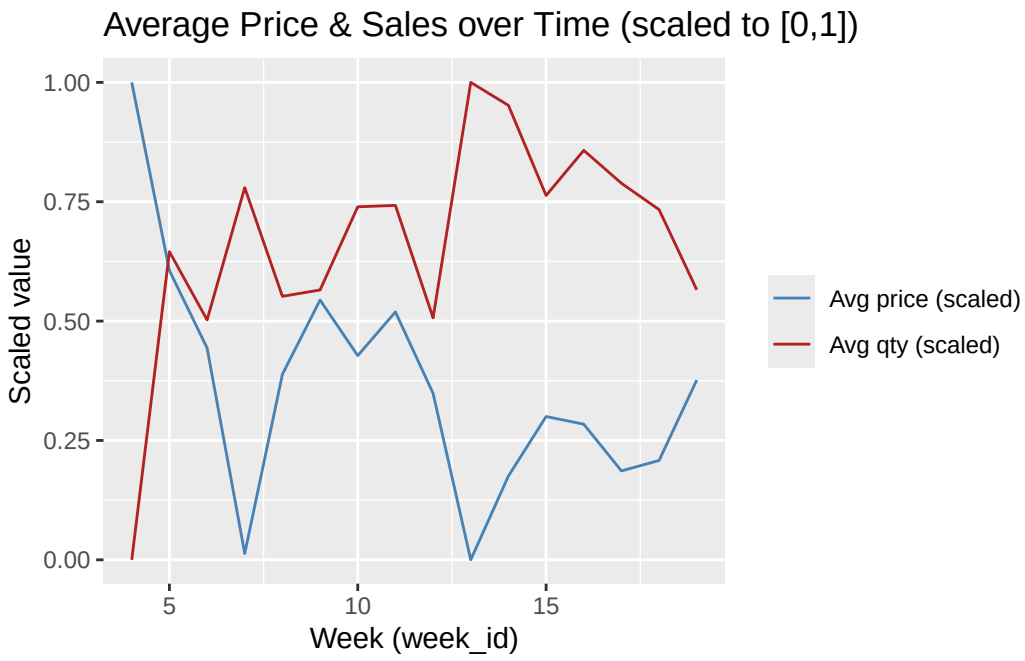



```
p_qty
```




```
# Save figures
ggsave("avg_price.png", p_price, width = 10, height = 6, dpi = 300)
```

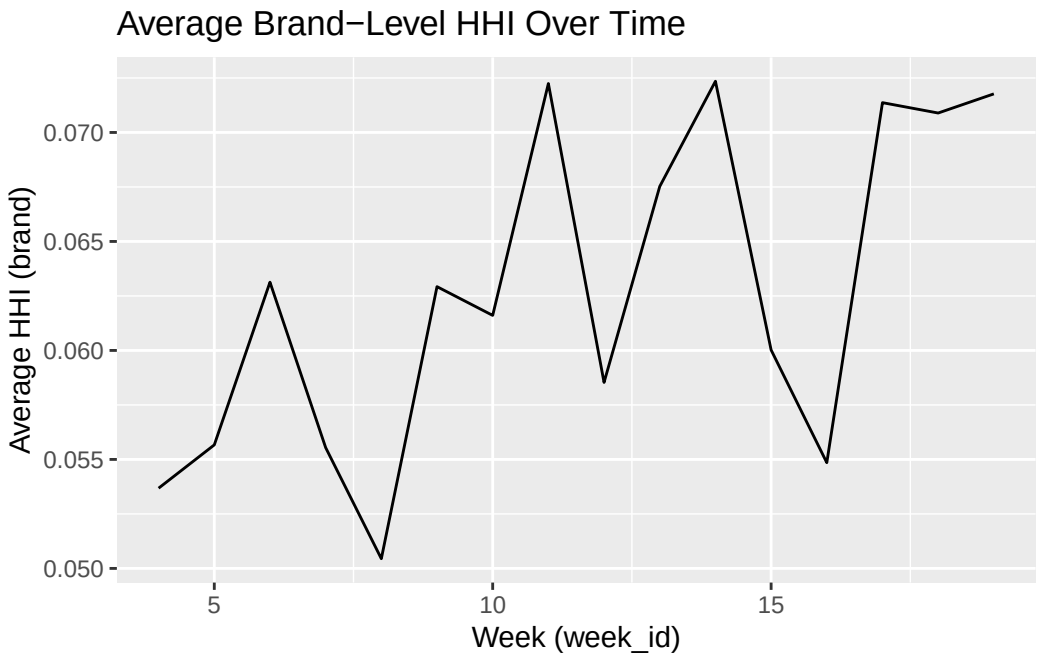
```r
ggsave("avg_sales.png", p_qty, width = 10, height = 6, dpi = 300)

# Dual-axis style (scaled)
ts_long_scaled <- ts_simple %>%
  mutate(
    price_scaled = (avg_price - min(avg_price, na.rm = TRUE)) /
                   (max(avg_price, na.rm = TRUE) - min(avg_price, na.rm = TRUE)),
    qty_scaled   = (avg_qty - min(avg_qty, na.rm = TRUE)) /
                   (max(avg_qty, na.rm = TRUE) - min(avg_qty, na.rm = TRUE))
  ) %>%
  select(week_id, price_scaled, qty_scaled) %>%
  pivot_longer(-week_id, names_to = "series", values_to = "value")

p_dual <- ggplot(ts_long_scaled, aes(x = week_id, y = value, color = series)) +
  geom_line() +
  scale_color_manual(values = c("price_scaled" = "steelblue", "qty_scaled" = "firebrick"),
                     labels = c("Avg price (scaled)", "Avg qty (scaled)")) +
  labs(title = "Average Price & Sales over Time (scaled to [0,1])",
       x = "Week (week_id)", y = "Scaled value", color = NULL)

p_dual
```



```r
ggsave("avg_price_sales_dual.png", p_dual, width = 10, height = 6, dpi = 300)
```

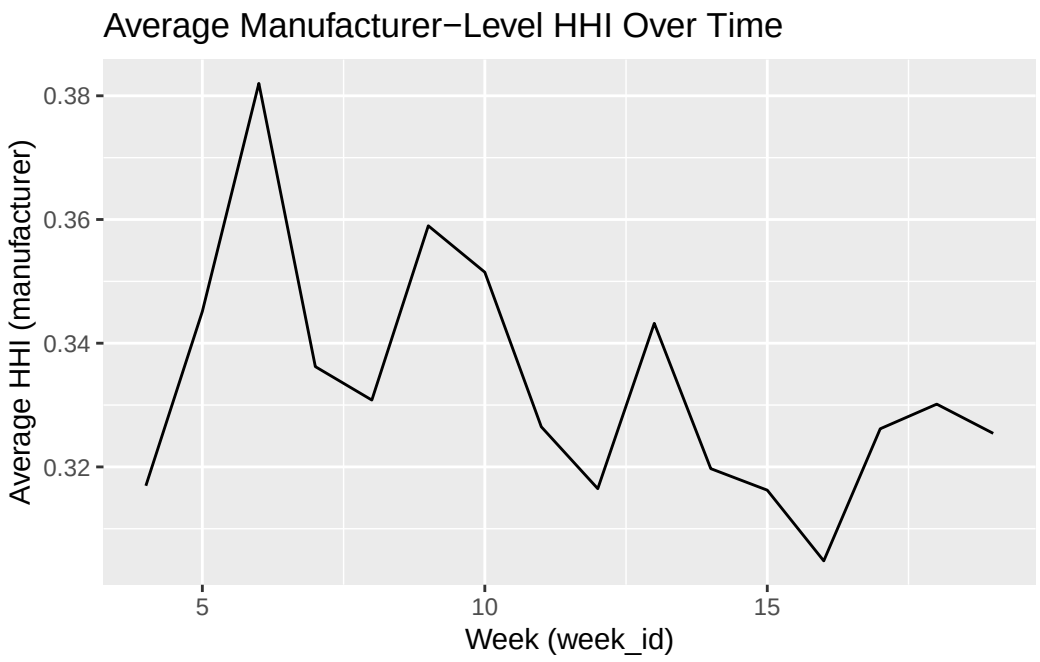
1.3

```r
# Brand HHI per market
brand_shares <- iri %>%
  group_by(market_id) %>%
  mutate(total_mkt_sales = sum(quantity, na.rm = TRUE)) %>%
  ungroup() %>%
  group_by(market_id, brand) %>%
  summarise(brand_sales = sum(quantity, na.rm = TRUE),
            total_mkt_sales = first(total_mkt_sales),
            .groups = "drop") %>%
  mutate(share_brand = if_else(total_mkt_sales > 0, brand_sales / total_mkt_sales, 0))

hhi_brand_market <- brand_shares %>%
  group_by(market_id) %>%
  summarise(HHI_brand = sum(share_brand^2, na.rm = TRUE), .groups = "drop")

hhi_brand_week <- iri %>%
  distinct(market_id, week_id) %>%
  inner_join(hhi_brand_market, by = "market_id") %>%
  group_by(week_id) %>%
  summarise(avg_HHI_brand = mean(HHI_brand, na.rm = TRUE), .groups = "drop") %>%
  arrange(week_id)

p_hhi_brand <- ggplot(hhi_brand_week, aes(x = week_id, y = avg_HHI_brand)) +
  geom_line() +
  labs(title = "Average Brand-Level HHI Over Time",
       x = "Week (week_id)", y = "Average HHI (brand)")

p_hhi_brand
```

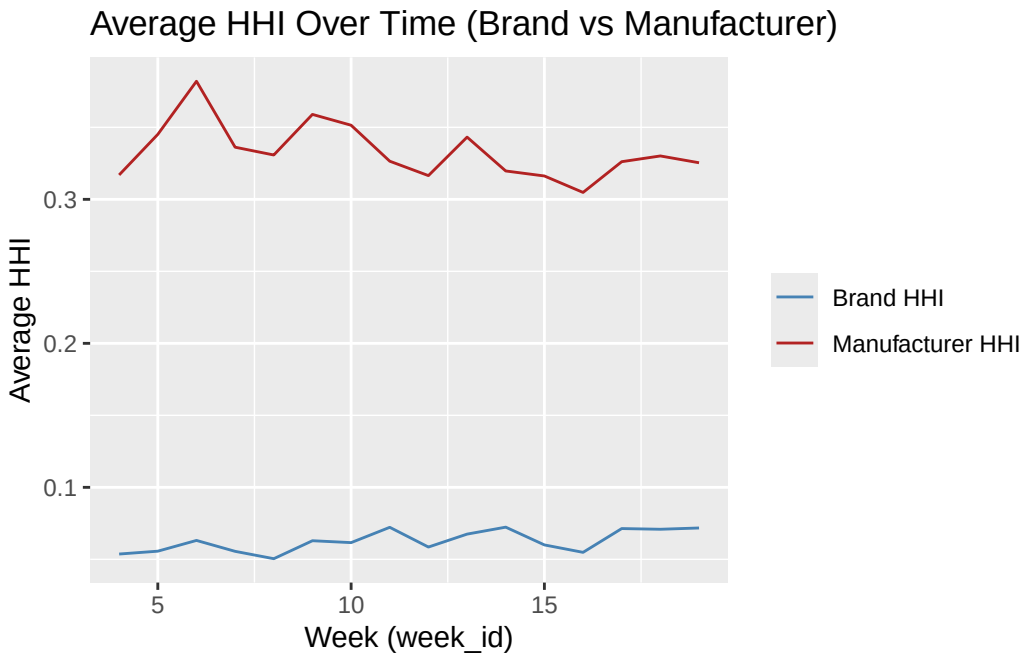



```
ggsave("HHI_brand.png", p_hhi_brand, width = 10, height = 6, dpi = 300)

# Manufacturer (parent) HHI per market
parent_shares <- iri %>%
  group_by(market_id) %>%
  mutate(total_mkt_sales = sum(quantity, na.rm = TRUE)) %>%
  ungroup() %>%
  group_by(market_id, parent) %>%
  summarise(parent_sales = sum(quantity, na.rm = TRUE),
            total_mkt_sales = first(total_mkt_sales),
            .groups = "drop") %>%
  mutate(share_parent = if_else(total_mkt_sales > 0, parent_sales / total_mkt_sales, 0))

hhi_parent_market <- parent_shares %>%
  group_by(market_id) %>%
  summarise(HHI_parent = sum(share_parent^2, na.rm = TRUE), .groups = "drop")

hhi_parent_week <- iri %>%
  distinct(market_id, week_id) %>%
  inner_join(hhi_parent_market, by = "market_id") %>%
  group_by(week_id) %>%
  summarise(avg_HHI_parent = mean(HHI_parent, na.rm = TRUE), .groups = "drop") %>%
  arrange(week_id)
```

```r
p_hhi_parent <- ggplot(hhi_parent_week, aes(x = week_id, y = avg_HHI_parent)) +
  geom_line() +
  labs(title = "Average Manufacturer-Level HHI Over Time",
       x = "Week (week_id)", y = "Average HHI (manufacturer)")

p_hhi_parent
```




```r
ggsave("HHI_parent.png", p_hhi_parent, width = 10, height = 6, dpi = 300)

# Combined comparison
hhi_both <- hhi_brand_week %>% inner_join(hhi_parent_week, by = "week_id")

p_hhi_both <- ggplot(hhi_both, aes(x = week_id)) +
  geom_line(aes(y = avg_HHI_brand, color = "Brand HHI")) +
  geom_line(aes(y = avg_HHI_parent, color = "Manufacturer HHI")) +
  scale_color_manual(values = c("Brand HHI" = "steelblue", "Manufacturer HHI" = "firebrick")) +
  labs(title = "Average HHI Over Time (Brand vs Manufacturer)",
       x = "Week (week_id)", y = "Average HHI", color = NULL)

p_hhi_both
```




```
ggsave("HHI_brand_vs_parent.png", p_hhi_both, width = 10, height = 6, dpi = 300)
```

Question 2:

2.1

```
# Prepare data for regression
reg_data <- iri %>%
  group_by(market_id) %>%
  mutate(
    market_fe = as.factor(store_id),
    manufacturer_fe = as.factor(parent),
    time_fe = as.factor(week_id),
    brand_fe = as.factor(brand),
    p_jmt = price,
    total_sales = sum(quantity, na.rm = TRUE),
    sj = quantity / M,
    s0 = 1 - (total_sales / M),
    u = log(sj) - log(s0)
  ) %>%
  ungroup()
```

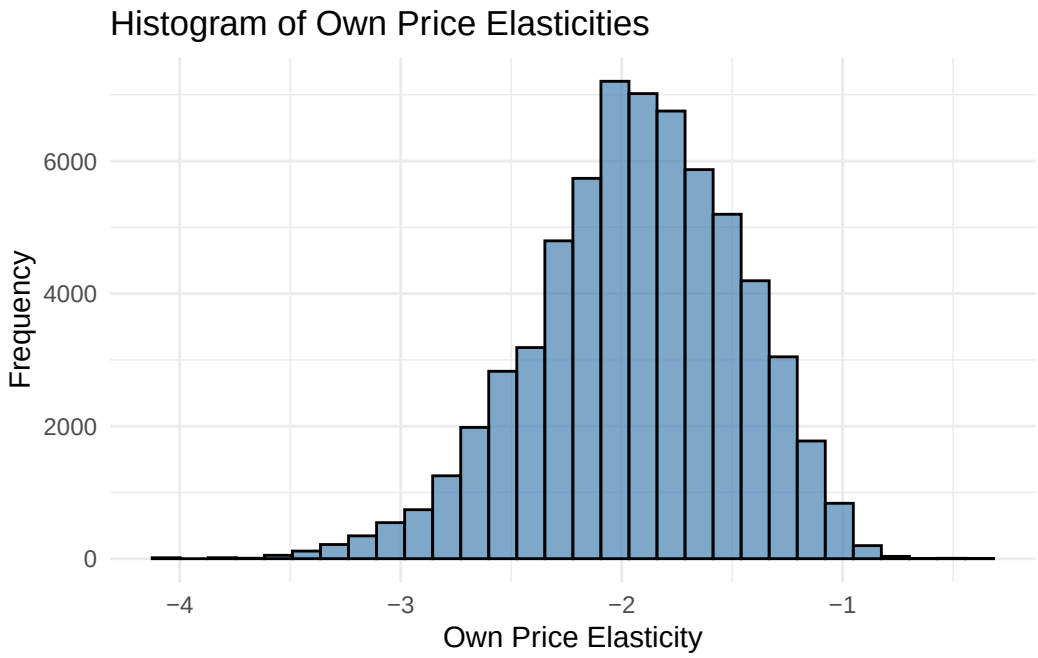
Our estimation model is thus:

$$u_{jmt} = \alpha p_{jmt} + \beta \mathbb{X}_{jmt} + \xi_{jmt}$$

where $\mathbb{X}_{jmt}$ includes the product characteristics fiber, sugars, flavored, and fortified, as well as fixed effects for market, manufacturer, and time. and u_{jmt} is the log-odds of product j's market share in market m at time t.

2.2

```
# Regression model
list <- c("fiber", "sugars", "flavored", "fortified",
          "market_fe", "manufacturer_fe", "time_fe")
reg_model <- feols(
  u ~ p_jmt + fiber + sugars + flavored + fortified |
  market_fe + manufacturer_fe + time_fe,
  data = reg_data
)

# Histogram of own price elasticities
reg_data <- reg_data %>%
  mutate(own_price_elasticity = coef(reg_model)["p_jmt"] * p_jmt * (1 - sj))

ggplot(reg_data, aes(x = own_price_elasticity)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black", alpha = 0.7) +
  labs(title = "Histogram of Own Price Elasticities",
       x = "Own Price Elasticity", y = "Frequency") +
  theme_minimal()
```



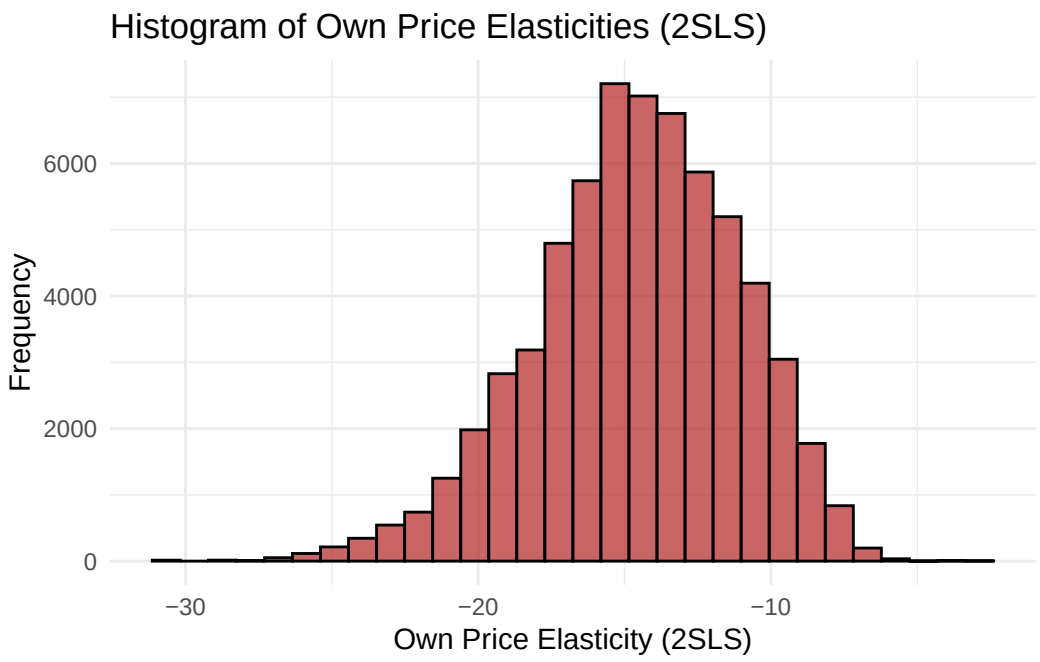
2.3

```
# 2SLS with Price Instrument: Price*Quantity of Sugar

reg_data <- reg_data %>%
  mutate(
    z = sugars * sugar_price,
    z = log(z)

  )
stage2 <- feols(u ~ fiber + sugars + flavored + fortified |
                  market_fe + manufacturer_fe + time_fe | p_jmt ~ z,
                data = reg_data)
summary(stage2)
```

```
TSLS estimation - Dep. Var.: u
                  Endo.    : p_jmt
                  Instr.   : z
Second stage: Dep. Var.: u
Observations: 63,966
Fixed-effects: market_fe: 642,  manufacturer_fe: 4,  time_fe: 16
```

```
Standard-errors: IID
             Estimate Std. Error   t value   Pr(>|t|)
fit_p_jmt -62.766445   7.898427  -7.94670 1.9468e-15 ***
fiber      -0.570755   0.066161  -8.62678  < 2.2e-16 ***
sugars     -0.120572   0.011575 -10.41695  < 2.2e-16 ***
flavored    0.179014   0.023118   7.74347 9.8166e-15 ***
fortified   0.377270   0.059710   6.31838 2.6607e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 2.49435     Adj. R2: 0.137296
                 Within R2: 0.025342
F-test (1st stage), p_jmt: stat =  54.680, p = 1.436e-13, on 1 and 63,301 DoF.
                Wu-Hausman: stat = 367.908, p < 2.2e-16  , on 1 and 63,300 DoF.
```

```r
# Histogram of own price elasticities (2SLS)
reg_data <- reg_data %>%
  mutate(own_price_elasticity_2sls = coef(stage2)["fit_p_jmt"] * p_jmt * (1 - sj))
ggplot(reg_data, aes(x = own_price_elasticity_2sls)) +
  geom_histogram(bins = 30, fill = "firebrick", color = "black", alpha = 0.7) +
  labs(title = "Histogram of Own Price Elasticities (2SLS)",
       x = "Own Price Elasticity (2SLS)", y = "Frequency") +
  theme_minimal()
```



Using 2SLS results in estimates which are more negative than the OLS estimates, indicating that the OLS estimates were biased towards zero.

2.4

```
# Elasticity Matrix for Top 5 Brands by Sales
## Create average matrix by taking average of market level elaticity matrices

top_brands <- reg_data %>%
  group_by(brand) %>%
  summarise(total_sales = sum(quantity, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(total_sales)) %>%
  slice_head(n = 5) %>%
  pull(brand)
elasticity_matrices <- list()
for (mkt in unique(reg_data$market_id)) {
  mkt_data <- reg_data %>% filter(market_id == mkt, brand %in% top_brands)
  n <- nrow(mkt_data)
  if (n < 5) next  # Skip markets with fewer than 5 top brands
  elas_matrix <- matrix(0, nrow = n, ncol = n)
  rownames(elas_matrix) <- mkt_data$brand
  colnames(elas_matrix) <- mkt_data$brand
  for (i in 1:n) {
    for (j in 1:n) {
      if (i == j) {
        elas_matrix[i, j] <- coef(stage2)["fit_p_jmt"] * mkt_data$p_jmt[i] * (1 - mkt_data$s
      } else {
        elas_matrix[i, j] <- -coef(stage2)["fit_p_jmt"] * mkt_data$p_jmt[j] * mkt_data$sj[i]
      }
    }
  }
  elasticity_matrices[[as.character(mkt)]] <- elas_matrix
}
# Average elasticity matrix
avg_elasticity_matrix <- Reduce("+", elasticity_matrices) / length(elasticity_matrices)
avg_elasticity_matrix
```

```
                           KELLOGGS FROSTED MINI WHEATS
KELLOGGS FROSTED MINI WHEATS                   -13.1423398
GENERAL MILLS CHEERIOS                           0.2615167
```

```
GENERAL MILLS HONEY NUT CHEER                    0.2654592
POST HONEY BUNCHES OF OATS                       0.2657008
KELLOGGS FROSTED FLAKES                          0.2576447
                            GENERAL MILLS CHEERIOS
KELLOGGS FROSTED MINI WHEATS              0.2638177
GENERAL MILLS CHEERIOS                  -13.2669948
GENERAL MILLS HONEY NUT CHEER             0.2671759
POST HONEY BUNCHES OF OATS                0.2681528
KELLOGGS FROSTED FLAKES                   0.2604532
                            GENERAL MILLS HONEY NUT CHEER
KELLOGGS FROSTED MINI WHEATS                      0.2650948
GENERAL MILLS CHEERIOS                            0.2639448
GENERAL MILLS HONEY NUT CHEER                   -13.2314072
POST HONEY BUNCHES OF OATS                        0.2691572
KELLOGGS FROSTED FLAKES                           0.2576187
                            POST HONEY BUNCHES OF OATS
KELLOGGS FROSTED MINI WHEATS                  0.2623082
GENERAL MILLS CHEERIOS                        0.2636535
GENERAL MILLS HONEY NUT CHEER                 0.2664261
POST HONEY BUNCHES OF OATS                  -13.2162577
KELLOGGS FROSTED FLAKES                       0.2599669
                            KELLOGGS FROSTED FLAKES
KELLOGGS FROSTED MINI WHEATS                0.2669329
GENERAL MILLS CHEERIOS                      0.2632467
GENERAL MILLS HONEY NUT CHEER               0.2654629
POST HONEY BUNCHES OF OATS                  0.2690599
KELLOGGS FROSTED FLAKES                   -13.2852126
```

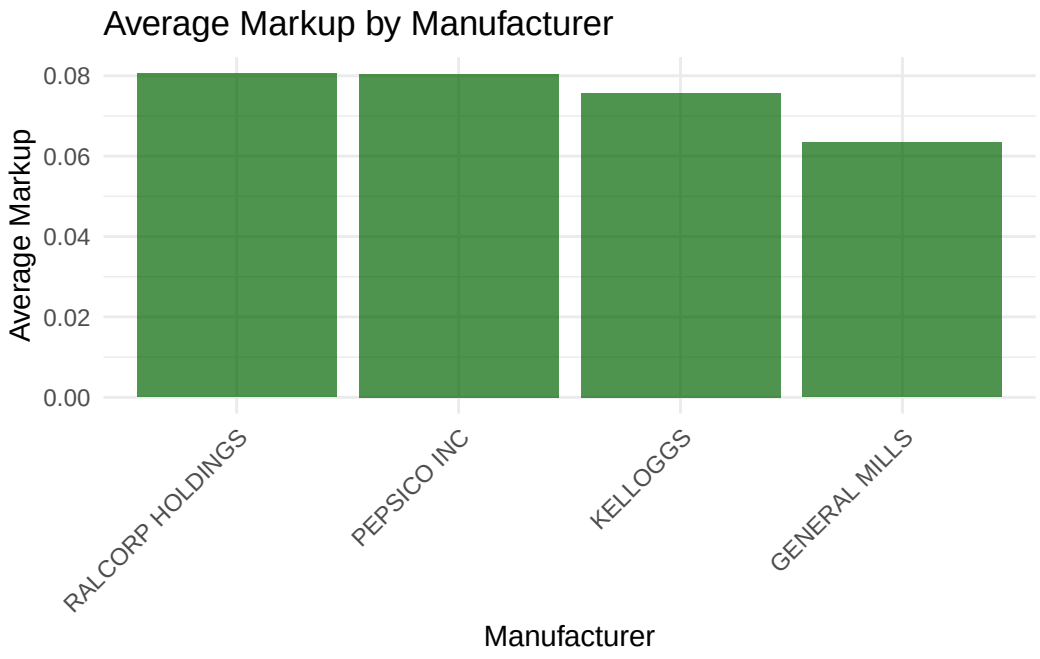
2.5

```r
# Calculaete markups for each manufacturer
reg_data <- reg_data %>%
  mutate(
    own_price_elasticity_2sls = coef(stage2)["fit_p_jmt"] * p_jmt * (1 - sj),
    markup = -1 / own_price_elasticity_2sls
  )
markup_summary <- reg_data %>%
  group_by(parent) %>%
  summarise(
    avg_markup = mean(markup, na.rm = TRUE),
    sd_markup = sd(markup, na.rm = TRUE),
```

```
    median_markup = median(markup, na.rm = TRUE),
    max_markup = max(markup, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(avg_markup))
markup_summary
```

```
# A tibble: 4 x 5
  parent           avg_markup sd_markup median_markup max_markup
  <chr>                 <dbl>     <dbl>         <dbl>      <dbl>
1 RALCORP HOLDINGS     0.0806    0.0225        0.0764      0.320
2 PEPSICO INC          0.0805    0.0179        0.0786      0.226
3 KELLOGGS             0.0757    0.0182        0.0727      0.182
4 GENERAL MILLS        0.0634    0.0127        0.0627      0.269
```

```
ggplot(markup_summary, aes(x = reorder(parent, -avg_markup), y = avg_markup)) +
  geom_bar(stat = "identity", fill = "darkgreen", alpha = 0.7) +
  labs(title = "Average Markup by Manufacturer",
       x = "Manufacturer", y = "Average Markup") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



2.6

```
# Nested by segment - construct data and calculate conditional shares
reg_data <- reg_data %>%
  mutate(
    nest = as.factor(segment)
  ) %>%
  group_by(market_id, nest) %>%
  mutate(
    total_nest_sales = sum(quantity, na.rm = TRUE),
    share_con = if_else(total_nest_sales > 0, quantity / total_nest_sales, 0)
  ) %>%
  ungroup()
```

The estimating equation is now:

$$u_{jmt} = \alpha p_{jmt} + \beta \mathbb{X}_{jmt} + \sigma_g s_{j|g} + \xi_{jmt}$$

where $s_{j|g}$ is the conditional share of product j within its nest g.

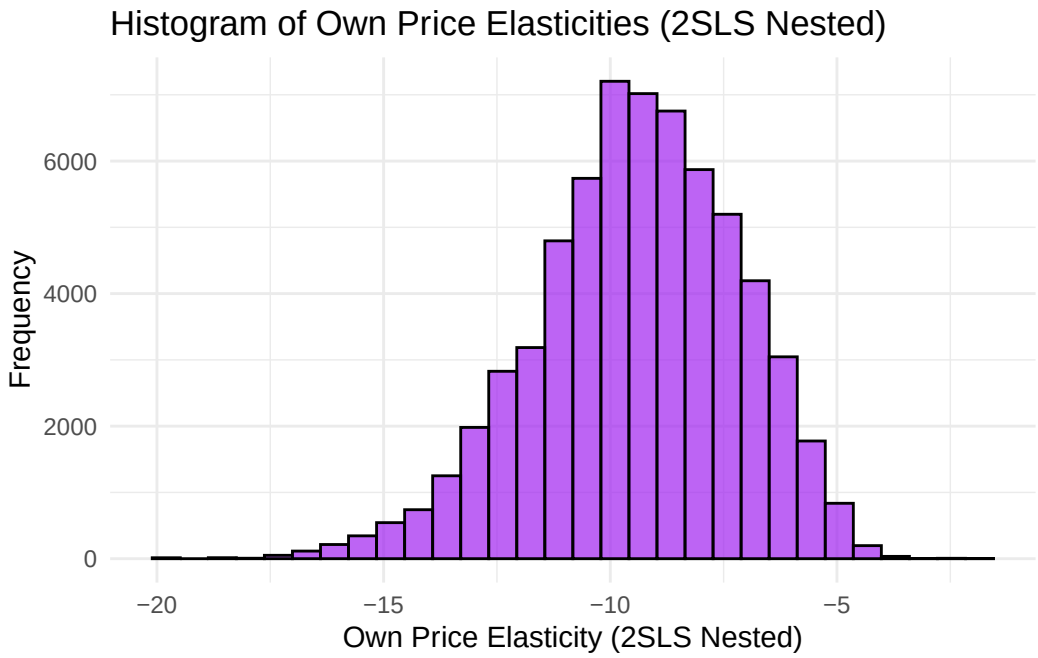
2.7

```
#2SLS with Price Instrument: Price*Quantity of Sugar and Conditional Share Instrument total p

reg_data <- reg_data %>%
  mutate(
    z_cond = as.numeric(nest) * (total_nest_sales / M),
    z = sugars * sugar_price,
    z_cond = log(z_cond),
    z = log(z)

  )
s2_nest <- feols(u ~ fiber + sugars + flavored + fortified |
                   market_fe + manufacturer_fe + time_fe | p_jmt + share_con ~ z + z_cond,
                 data = reg_data)
summary(s2_nest)
```

```
TSLS estimation - Dep. Var.: u
                  Endo.    : p_jmt, share_con
```

```
                  Instr.    : z, z_cond
Second stage: Dep. Var.: u
Observations: 63,966
Fixed-effects: market_fe: 642,  manufacturer_fe: 4,  time_fe: 16
Standard-errors: IID
                Estimate Std. Error   t value   Pr(>|t|)
fit_p_jmt     -40.527865   1.356599 -29.87461  < 2.2e-16 ***
fit_share_con   3.705776   0.671208   5.52106 3.3829e-08 ***
fiber          -0.353688   0.009536 -37.08915  < 2.2e-16 ***
sugars         -0.077466   0.002345 -33.03325  < 2.2e-16 ***
flavored        0.222589   0.016446  13.53458  < 2.2e-16 ***
fortified       0.171498   0.017644   9.72009  < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 1.63179     Adj. R2: 0.200364
                Within R2: 0.09661
F-test (1st stage), p_jmt    : stat =  1,090.9, p < 2.2e-16, on 2 and 63,300 DoF.
F-test (1st stage), share_con: stat =  1,328.2, p < 2.2e-16, on 2 and 63,300 DoF.
                   Wu-Hausman: stat = 13,814.9, p < 2.2e-16, on 2 and 63,298 DoF.
```

```r
# Histogram of own price elasticities (2SLS Nested)
reg_data <- reg_data %>%
  mutate(own_price_elasticity_2sls_nest = coef(s2_nest)["fit_p_jmt"] * p_jmt * (1 - sj))
ggplot(reg_data, aes(x = own_price_elasticity_2sls_nest)) +
  geom_histogram(bins = 30, fill = "purple", color = "black", alpha = 0.7) +
  labs(title = "Histogram of Own Price Elasticities (2SLS Nested)",
       x = "Own Price Elasticity (2SLS Nested)", y = "Frequency") +
  theme_minimal()
```



2.8

```r
# Elasticity Matrix for Top 5 Brands by Sales (Nested)
top_nest_brands <- reg_data %>%
  group_by(brand) %>%
  summarise(total_sales = sum(quantity, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(total_sales)) %>%
  slice_head(n = 5) %>%
  pull(brand)
elasticity_nest_matrices <- list()
for (mkt in unique(reg_data$market_id)) {
  mkt_data <- reg_data %>% filter(market_id == mkt, brand %in% top_nest_brands)
  n <- nrow(mkt_data)
  if (n < 5) next  # Skip markets with fewer than 5 top brands
  elas_matrix_nest <- matrix(0, nrow = n, ncol = n)
  rownames(elas_matrix_nest) <- mkt_data$brand
  colnames(elas_matrix_nest) <- mkt_data$brand
  for (i in 1:n) {
    for (j in 1:n) {
      if (i == j) {
        elas_matrix_nest[i, j] <- coef(s2_nest)["fit_p_jmt"] * mkt_data$p_jmt[i] * (1 - mkt_
      } else if (mkt_data$nest[i] == mkt_data$nest[j]) {
```

```
        elas_matrix_nest[i, j] <- -coef(s2_nest)["fit_p_jmt"] * mkt_data$p_jmt[j] * (mkt_data
      } else {
        elas_matrix_nest[i, j] <- -coef(s2_nest)["fit_p_jmt"] * mkt_data$p_jmt[j] * (mkt_data
      }
    }
  }
  elasticity_nest_matrices[[as.character(mkt)]] <- elas_matrix_nest
}
# Average elasticity matrix (Nested)
avg_elasticity_nest_matrix <- Reduce("+", elasticity_nest_matrices) /
  length(elasticity_nest_matrices)
avg_elasticity_nest_matrix
```

```
                            KELLOGGS FROSTED MINI WHEATS
KELLOGGS FROSTED MINI WHEATS                   -8.4859191
GENERAL MILLS CHEERIOS                          0.1335324
GENERAL MILLS HONEY NUT CHEER                   0.1353385
POST HONEY BUNCHES OF OATS                      0.1366943
KELLOGGS FROSTED FLAKES                         0.1329018
                            GENERAL MILLS CHEERIOS
KELLOGGS FROSTED MINI WHEATS              0.1349876
GENERAL MILLS CHEERIOS                   -8.5664081
GENERAL MILLS HONEY NUT CHEER             0.1362971
POST HONEY BUNCHES OF OATS                0.1378150
KELLOGGS FROSTED FLAKES                   0.1344631
                            GENERAL MILLS HONEY NUT CHEER
KELLOGGS FROSTED MINI WHEATS                     0.1350697
GENERAL MILLS CHEERIOS                           0.1348523
GENERAL MILLS HONEY NUT CHEER                   -8.5434294
POST HONEY BUNCHES OF OATS                       0.1381967
KELLOGGS FROSTED FLAKES                          0.1331539
                            POST HONEY BUNCHES OF OATS
KELLOGGS FROSTED MINI WHEATS                  0.1341761
GENERAL MILLS CHEERIOS                        0.1344155
GENERAL MILLS HONEY NUT CHEER                 0.1355287
POST HONEY BUNCHES OF OATS                   -8.5336475
KELLOGGS FROSTED FLAKES                       0.1341644
                            KELLOGGS FROSTED FLAKES
KELLOGGS FROSTED MINI WHEATS               0.1359956
GENERAL MILLS CHEERIOS                     0.1346289
GENERAL MILLS HONEY NUT CHEER              0.1357517
POST HONEY BUNCHES OF OATS                 0.1381995
```

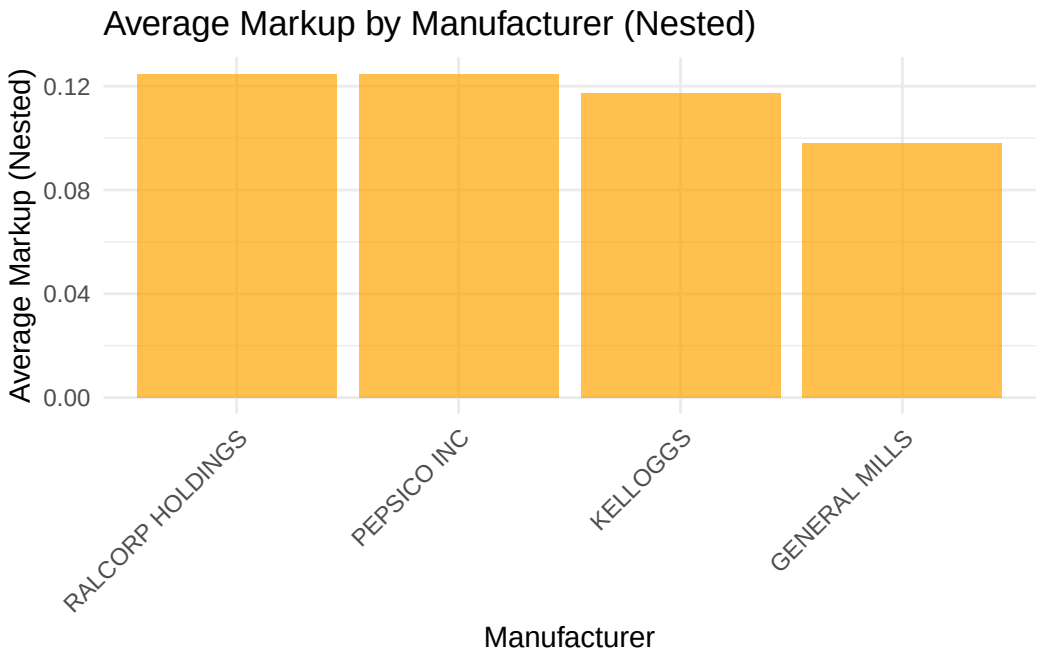
```
KELLOGGS FROSTED FLAKES                    -8.5781712
```

2.9

```
# Calculate average markups for each manufacturer (Nested)
reg_data <- reg_data %>%
  mutate(
    ope_nest = coef(s2_nest)["fit_p_jmt"] * p_jmt * (1 - sj),
    markup_nest = -1 / ope_nest
  )
markup_nest_summary <- reg_data %>%
  group_by(parent) %>%
  summarise(
    avg_markup_nest = mean(markup_nest, na.rm = TRUE),
    sd_markup_nest = sd(markup_nest, na.rm = TRUE),
    median_markup_nest = median(markup_nest, na.rm = TRUE),
    max_markup_nest = max(markup_nest, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(avg_markup_nest))
markup_nest_summary
```

```
# A tibble: 4 x 5
  parent       avg_markup_nest sd_markup_nest median_markup_nest max_markup_nest
  <chr>                  <dbl>          <dbl>              <dbl>           <dbl>
1 RALCORP HOL~           0.125         0.0348              0.118           0.496
2 PEPSICO INC            0.125         0.0277              0.122           0.350
3 KELLOGGS               0.117         0.0282              0.113           0.282
4 GENERAL MIL~           0.0982        0.0197              0.0971          0.417
```

```
ggplot(markup_nest_summary, aes(x = reorder(parent, -avg_markup_nest),
                                y = avg_markup_nest)) +
  geom_bar(stat = "identity", fill = "orange", alpha = 0.7) +
  labs(title = "Average Markup by Manufacturer (Nested)",
       x = "Manufacturer", y = "Average Markup (Nested)") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



2.10

```
print("HI")
```

```
[1] "HI"
```